

UJIAN AKHIR SEMESTER GENAP

IF260 - Sistem Operasi (Teori)

Nama: Kafijaya

NIM: 0000061651

1. Soal 1

Selesaikan permasalahan berikut ini menggunakan Banker's algorithm! Tentukan apakah sistem dalam kondisi safe state? Apa saja safe sequence-nya?

Process	Allocation				Max				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	0	1	0	0	0	3	2	5	2	4	1	2
P1	1	0	1	1	4	9	1	4				
P2	7	0	0	0	9	5	5	7				
P3	0	5	0	2	1	5	0	4				

Jawab:

R:

P0: 0 2 3 5

P1: 3 9 0 3

P2: 2 5 5 7

P3: 1 0 0 2

Available:

1) -> P3: 2 4 1 2
 1 0 0 2
 -----+
 3 4 1 4

2) -> NONE (Tidak ada available process yang tersisa)

Untuk safe sequence, hanya ada $r = \{P3\}$ karena hanya itu yang bisa dialokasikan proses tanpa deadlock. Namun, setelah itu, tidak ada lagi yang bisa di alokasikan setelah P3 selesai. Setelah itu semua menjadi deadlock

Penjelasan:

Sistem tidak berada dalam kondisi safe state karena system mengalami deadlock karena tidak memiliki resource yang cukup untuk menyelesaikan program. Hal ini dapat dilihat ketika system selesai menjalankan proses 3. Sistem memiliki resource A sebanyak 2, B sebanyak 9, C sebanyak 1, dan D sebanyak 4. Resource ini tidak cukup untuk dialokasikan kepada proses 0, 1, dan 2. Proses 0 memerlukan resource C sebanyak 2 dan D sebanyak 5 sedangkan hanya dimiliki resource C sebanyak 1 dan D sebanyak 4. Proses 1 memerlukan resource A sebanyak 3 sedangkan hanya dimiliki resource A sebanyak 2. Proses 2 memerlukan resource C sebanyak 5 dan D sebanyak 7 sedangkan hanya dimiliki resource C sebanyak 1 dan D sebanyak 4. Serta safe sequence yang terjadi dalam system adalah P3 dimana resource yang dimiliki adalah A sebanyak 2, B sebanyak 4, C sebanyak 1, dan D sebanyak 2 dan P3 membutuhkan resource A sebanyak 1 dan D sebanyak 2 sehingga P3 dapat dijalankan dan berada dalam safe state.

2. Soal 2

Selesaikan permasalahan memory compaction menggunakan Memory Management algorithm sebagai berikut. Tampilkan hasil alokasi akhir partisi berdasarkan tiap-tiap algoritma dan ketentuan soal yang ada!

- a. next-fit?
- b. best-fit?
- c. quick-fit?

ProcessSize[] = {20, 50, 4, 5, 10, 80, 81, 14}

blockSize[] =

20KB	14KB	10KB	100KB	7KB	10KB	5KB	20KB	44KB	81KB
------	------	------	-------	-----	------	-----	------	------	------

Jawab:

a. next-fit

- i. 20: Block 0 (20KB)
- ii. 50: Block 3 (100KB)
- iii. 4: Block 4 (7KB)
- iv. 5: Block 5 (10KB)
- v. 10: Block 7 (20KB)
- vi. 80: Block 9 (81KB)
- vii. 81: UNALLOCATED
- viii. 14: Block 1 (14KB)

sisProcessSize[] = {81}

Blocks: | 0 | 0 | 10 | 50 | 3 | 5 | 5 | 10 | 33 | 1 |

b. best-fit

- i. 20: Block 0 (20KB)
- ii. 50: Block 3 (100KB)
- iii. 4: Block 6 (5KB)
- iv. 5: Block 4 (7KB)
- v. 10: Block 2 (10KB)
- vi. 80: Block 9 (81KB)
- vii. 81: UNALLOCATED
- viii. 14: Block 1 (14KB)

sisProcessSize[] = {81}

Blocks: | 0 | 0 | 0 | 50 | 2 | 10 | 1 | 20 | 44 | 1 |

c. quick-fit

- i. 20: Block 0 (20KB)
- ii. 50: Block 3 (100KB)
- iii. 4: Block 1 (14KB)
- iv. 5: Block 1 (14KB)
- v. 10: Block 2 (10KB)
- vi. 80: Block 9 (81KB)
- vii. 81: UNALLOCATED
- viii. 14: Block 7 (20KB)

sisProcessSize[] = {81}

Blocks: | 0 | 5 | 0 | 50 | 7 | 10 | 5 | 6 | 44 | 1 |

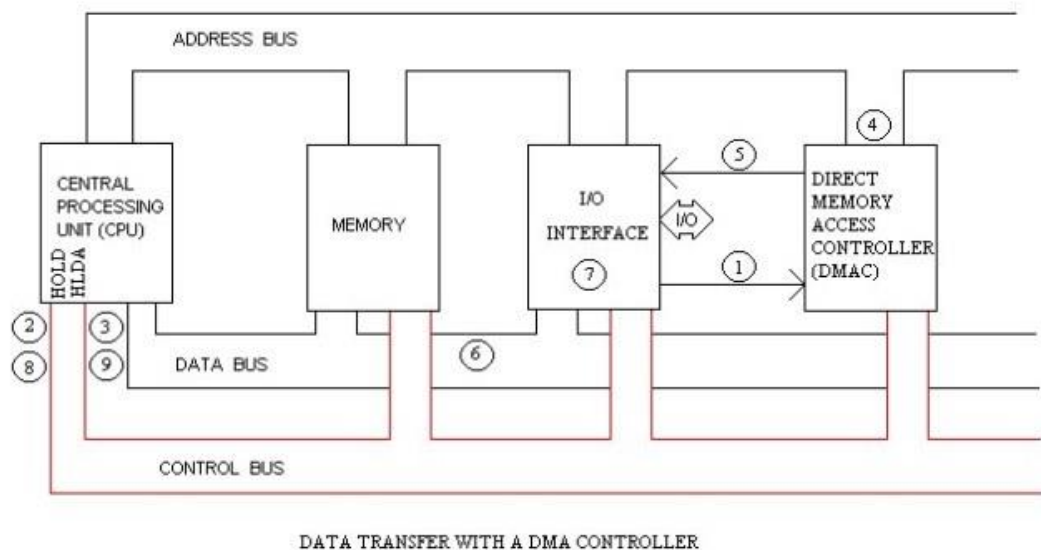
3. Soal 3

- a. Jelaskan dan ilustrasikan fungsi dari DMA Controller!

Jawab:

DMA merupakan singkatan dari Direct Memory Access. Sesuai namanya, Direct Memory Access adalah sistem yang bertugas untuk memberikan akses langsung terhadap data tanpa perlu melalui proses di CPU. Dengan begitu, kamu dapat mempercepat operasi memori dan tidak memberatkan kerja CPU. Namun, hanya input dan output data tertentu saja yang dapat diproses melalui DMA.

Channel DMA digunakan untuk mengkomunikasikan data antara perangkat periferan dan memori sistem. Tanpa DMA, CPU menyalin setiap bagian data dan tidak memungkinkan pekerjaan lain dilakukan sampai operasi selesai. Dengan DMA, CPU dapat memproses tugas lain saat transfer data sedang dilakukan. Transfer data pertama kali diprakarsai oleh CPU, namun operasinya dapat dilakukan oleh DMA.



Fungsi dari DMA sendiri adalah agar CPU dapat melakukan pekerjaan atau instruksi yang berbeda ketika melakukan operasi baca tulis dari perangkat peripheral. Tanpa adanya DMA CPU akan terus sibuk melakukan operasi baca tulis (transfer data) dan tidak dapat melakukan atau menyelesaikan instruksi yang lain. Dengan adanya DMA, CPU cukup mempersiapkan DMA chip dengan cara memberikan beberapa informasi seperti jumlah data bit yang ditransfer, alamat dari device dan memori yang diperlukan dan arah dari aliran data tersebut, setelah itu DMA chip sendiri yang akan menyelesaikannya. DMA chip akan melakukan interrupt, ketika pekerjaannya sudah selesai. Selama DMA chip melakukan tugasnya hingga munculnya interrupt, CPU dapat menyelesaikan instruksi yang lainnya.

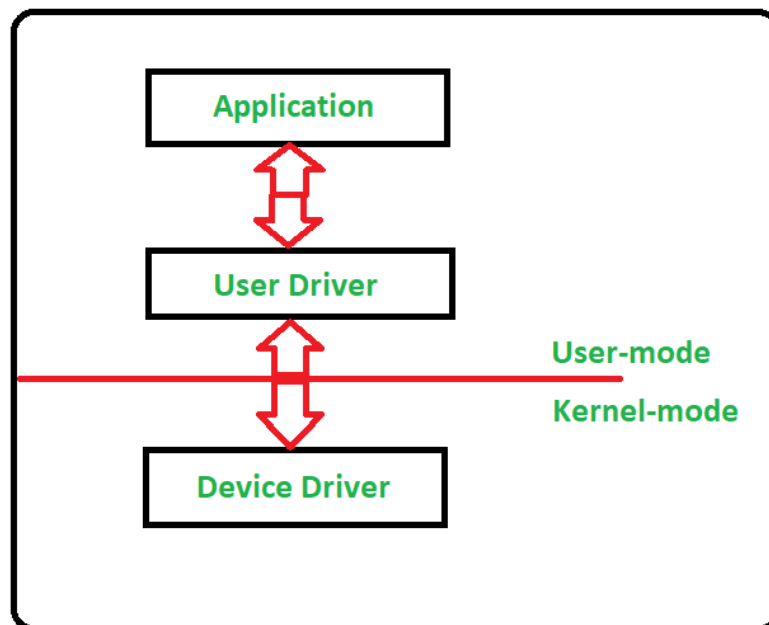
DMA juga digunakan intra-chip untuk transfer data dalam multi-core, terutama dalam sistem multiprocessor-on-chip, di mana elemen-nya adalah proses yang dilengkapi dengan memori lokal (sering disebut alas memori) dan DMA digunakan untuk mentransfer data antara lokal memori dan memori utama. Komputer yang ada DMA channel dapat mentransfer data dari dan ke perangkat dengan CPU overhead

jauh lebih sedikit daripada komputer tanpa saluran DMA . Demikian pula di dalam sebuah elemen pemrosesan multi-core processor dapat mentransfer data dari memori tanpa menempati prosesor waktu, mesin dan membuat data tumpang tindih.

b. Jelaskan dan ilustrasikan fungsi dari device driver!

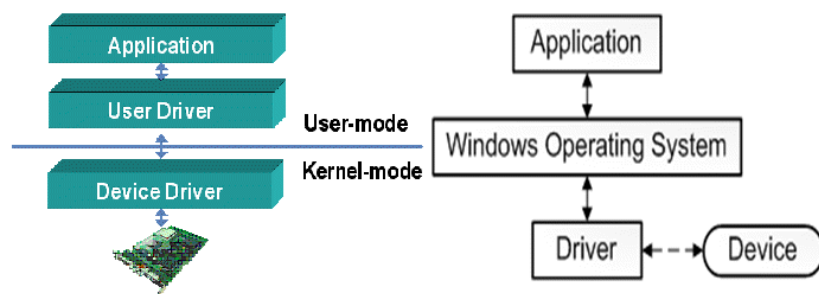
Jawab:

Device Driver adalah program jembatan penghubung antara hardware dan software untuk membantu mengontrol mengendalikan perangkat perifer komputer. Suatu perangkat perifer tidak akan berfungsi tanpa adanya device driver, kecuali perangkat perifer digunakan bersifat Build Up atau Genuine Parts menurut vendor Operating System (OS) yang digunakan. device driver biasanya menyediakan piranti lunak antar muka (tampilan) untuk memudahkan pengguna.



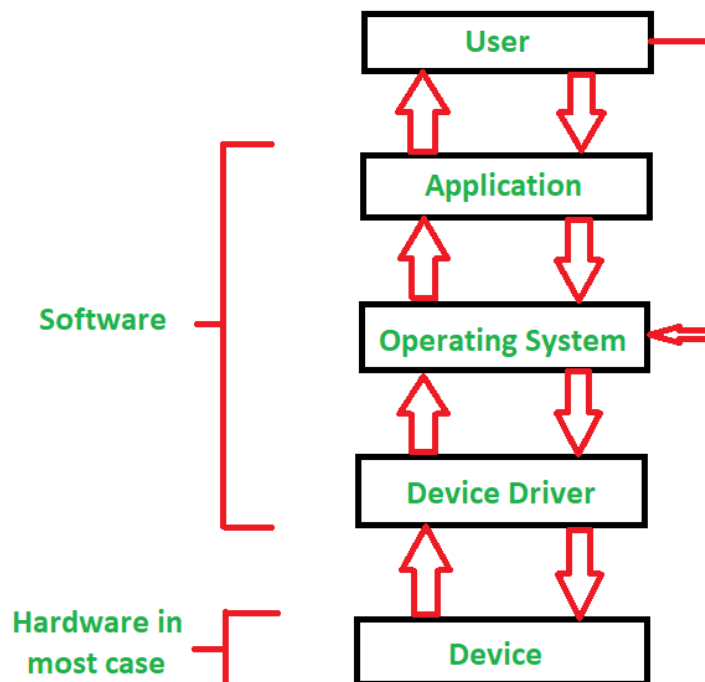
Fungsi utama dari device driver adalah sebagai penghubung utama hardware. device driver tidak akan berguna tanpa hardware begitu juga kebalikannya. Fungsi dari device driver lainnya adalah:

- Jembatan penghubung Software dengan hardware.
- Penerjemah data antara Software dan hardware.
- Pengontrol Input dan Output Software terhadap hardware.



Cara Kerja Device Driver :

Device Driver bergantung pada instruksi Sistem Operasi untuk mengakses perangkat dan melakukan tindakan tertentu. Setelah aksi, mereka juga menunjukkan reaksi mereka dengan mengirimkan output atau status/pesan dari perangkat keras ke sistem Operasi. Misalnya, driver printer memberi tahu printer dalam format apa yang akan dicetak setelah mendapatkan instruksi dari OS, demikian pula Driver kartu suara ada karena ke mana data 1 dan 0 dari file MP3 diubah menjadi sinyal audio dan Anda menikmati musiknya. Pembaca kartu, pengontrol, modem, kartu jaringan, kartu suara, printer, kartu video, perangkat USB, RAM, Speaker dll memerlukan Driver Perangkat untuk beroperasi.



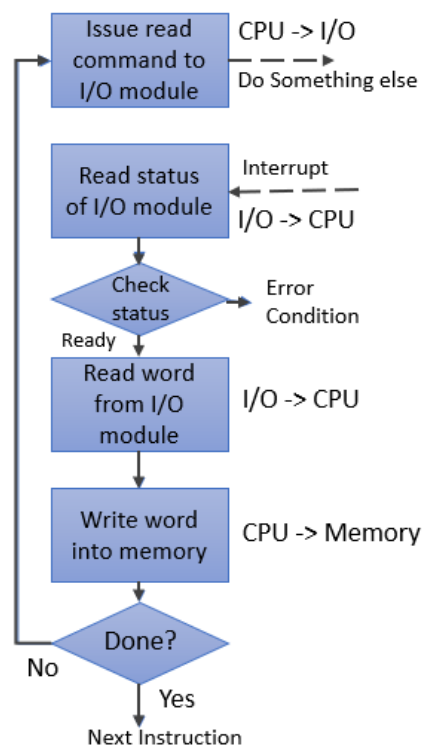
c. Jelaskan cara kerja interrupt-driven I/O?

Jawab:

Teknik interrupt – driven I/O memungkinkan proses tidak membuang – buang waktu. Prosesnya adalah CPU mengeluarkan perintah I/O pada modul I/O, bersamaan perintah I/O dijalankan modul I/O maka CPU akan melakukan eksekusi perintah – perintah lainnya. Apabila modul I/O telah selesai menjalankan instruksi yang diberikan padanya akan melakukan interupsi pada CPU bahwa tugasnya telah selesai.

Dalam teknik ini kendali perintah masih menjadi tanggung jawab CPU, baik pengambilan perintah dari memori maupun pelaksanaan isi perintah tersebut. Terdapat selangkah kemajuan *dari teknik sebelumnya, yaitu CPU melakukan multitasking beberapa perintah sekaligus sehingga* tidak ada waktu tunggu bagi CPU.

Cara kerja teknik interupsi di sisi modul I/O adalah modul I/O menerima perintah, misal *read*. Kemudian modul I/O melaksanakan perintah pembacaan dari peripheral dan meletakkan paket data ke register data modul I/O, selanjutnya modul mengeluarkan sinyal interupsi ke CPU melalui saluran kontrol. Kemudian modul menunggu datanya diminta CPU. Saat permintaan terjadi, modul meletakkan data pada bus data dan modul siap menerima perintah selanjutnya. Pengolahan interupsi saat perangkat I/O telah menyelesaikan sebuah operasi I/O adalah sebagai berikut :



Interrupted I/O to Transfer Data
from I/O Module to Memory

1. Perangkat I/O akan mengirimkan sinyal interupsi ke CPU.
2. CPU menyelesaikan operasi yang sedang dijalankannya kemudian merespon interupsi.
3. CPU memeriksa interupsi tersebut, kalau valid maka CPU akan mengirimkan sinyal acknowledgment ke perangkat I/O untuk menghentikan interupsinya.
4. CPU mempersiapkan pengontrolan transfer ke routine interupsi. Hal yang dilakukan adalah menyimpan informasi yang diperlukan untuk melanjutkan operasi yang tadi dijalankan sebelum adanya interupsi. Informasi yang diperlukan berupa:
 - a. Status processor, berisi register yang dipanggil PSW (program status word).
 - b. Lokasi intruksi berikutnya yang akan dieksekusi.
5. Informasi tersebut kemudian disimpan dalam stack pengontrol sistem.
6. Kemudian CPU akan menyimpan PC (program counter) eksekusi sebelum interupsi ke stack pengontrol bersama informasi PSW. Selanjutnya mempersiapkan PC untuk penanganan interupsi.
7. Selanjutnya CPU memproses interupsi sampai selesai.
8. Apabila pengolahan interupsi selesai, CPU akan memanggil kembali informasi yang telah disimpan pada stack pengontrol untuk meneruskan operasi sebelum interupsi.

Terdapat bermacam teknik yang digunakan CPU dalam menangani program interupsi ini, diantaranya:

- Multiple Interrupt Lines.
- Software poll.
- Daisy Chain.
- Arbitrasi bus.

Teknik yang paling sederhana adalah menggunakan saluran interupsi berjumlah banyak (Multiple Interrupt Lines) antara CPU dan modul – modul I/O. Namun tidak praktis untuk menggunakan sejumlah saluran bus atau pin CPU ke seluruh saluran interupsi modul – modul I/O.

Alternatif lainnya adalah menggunakan software poll. Prosesnya, apabila CPU mengetahui adanya sebuah interupsi, maka CPU akan menuju ke routine layanan interupsi yang tugasnya melakukan poll seluruh modul I/O untuk menentukan modul yang melakukan interupsi. Kerugian software poll adalah memerlukan waktu yang lama karena harus mengidentifikasi seluruh modul untuk mengetahui modul I/O yang melakukan interupsi.

Teknik yang lebih efisien adalah daisy chain, yang menggunakan hardware poll. Seluruh modul I/O tersambung dalam saluran interupsi CPU secara melingkar (chain). Apabila ada permintaan interupsi, maka CPU akan menjalankan sinyal acknowledge yang berjalan pada saluran interupsi sampai menjumpai modul I/O yang mengirimkan interupsi.

Teknik berikutnya adalah arbitrase bus. Dalam metode ini, pertama – tama modul I/O memperoleh kontrol bus sebelum modul ini menggunakan saluran permintaan interupsi. Dengan demikian hanya akan terdapat sebuah modul I/O yang dapat melakukan interupsi.

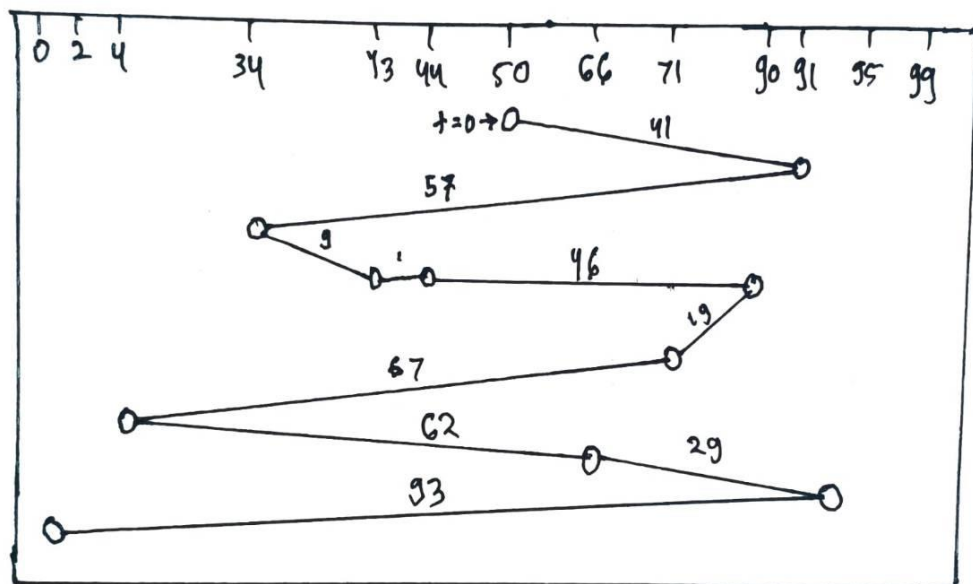
4. Soal 4

Selesaikan permasalahan disk scheduling menggunakan algoritma FCFS dengan posisi Read/Write head saat ini di track 50 dan permintaan akses sebagai berikut. Ilustrasikan pergerakan Read/Write head dari awal permintaan hingga akhir selesai dan hitung seek time totalnya!

91	34	43	44	90	71	4	66	95	2
----	----	----	----	----	----	---	----	----	---

Jawab:

91, 34, 43, 44, 90, 71, 4, 66, 95, 2



Total seek time =

$$41 + 57 + 9 + 1 + 46 + 19 + 67 + 62 + 29 + 93 = 424$$